ENTREGA MEMORIA DÍA 15 UNION-SHOP

Sumario

1. INTRODUCCIÓN....3
1.1. Descripción....3
1.2. Motivación....3
2. GESTIÓN Y ORGANIZACIÓN DEL PROYECTO....4
2.1. Metodología de desarrollo....4
2.2. Planificación....4
2.3. Ciclo de vida....5
3. OBJETIVOS....5
4. DESARROLLO DE LA APLICACIÓN....5
4.1. Análisis: requisitos....5
4.2. Diseño: arquitectura....7
4.3. Implementación: decisiones adoptadas y problemas encontrados....8
8. BIBLIOGRAFÍA....9
Publicaciones....9
Enlaces web....9
Enlace a GitHub....11
https://github.com/hectorrostro/Union-Shop....11

1. INTRODUCCIÓN

1.1. Descripción

Union-Shop es una aplicación desarrollada en Python que simula el funcionamiento de un centro comercial virtual. La aplicación permite al usuario navegar por las distintas tiendas disponibles, consultar productos, añadirlos a un carrito de la compra y finalizar la compra. Además, dispone de un modo administrador que permite gestionar el catálogo de productos, incluyendo añadir, modificar y eliminar artículos, con persistencia de datos mediante MongoDB y ficheros JSON como copia de seguridad.

El proyecto ha sido desarrollado por un equipo de tres alumnos del CFGM de Sistemas Microinformáticos y Redes como Proyecto de Fin de Ciclo, utilizando Python 3 como lenguaje de programación y Visual Studio Code como entorno de desarrollo.

1.2. Motivación

La principal motivación para realizar este proyecto ha sido la de aplicar de forma práctica los conocimientos de programación adquiridos a lo largo del ciclo formativo. Durante el curso hemos aprendido los fundamentos de la programación en Python y consideramos que la mejor manera de consolidar ese aprendizaje era desarrollando una aplicación real y funcional desde cero.

Elegimos la temática de un centro comercial virtual porque es algo cercano y familiar para cualquier persona. Las aplicaciones de compra online forman parte del día a día de millones de usuarios, lo que nos permitía tener una idea clara de cómo debía funcionar la aplicación y qué características debía incluir.

Además, este tipo de aplicación nos permitía poner en práctica conceptos clave trabajados en clase, como el manejo de estructuras de datos, la gestión de ficheros, la creación de menús interactivos y la lógica de negocio básica, todo ello dentro de un proyecto con sentido completo y coherente.

Por último, el hecho de trabajar en equipo ha sido también una motivación en sí misma, ya que nos ha permitido simular un entorno de desarrollo real, repartiendo

tareas y coordinando el trabajo entre los tres integrantes del grupo.

2. GESTIÓN Y ORGANIZACIÓN DEL PROYECTO

2.1. Metodología de desarrollo

Para el desarrollo de Union-Shop se ha seguido una metodología en cascada, en la que cada fase del proyecto se planificó y completó antes de pasar a la siguiente. En primer lugar se definieron los requisitos y objetivos del proyecto, posteriormente se diseñó la arquitectura de la aplicación, a continuación se llevó a cabo la implementación del código y finalmente se realizaron las pruebas.

Esta metodología fue la más adecuada para nuestro equipo dado que el alcance del proyecto estaba bien definido desde el inicio y el tamaño del grupo era reducido, lo que facilitaba la coordinación sin necesidad de metodologías más complejas.

2.2. Planificación

La planificación del proyecto se dividió en las siguientes fases:

Fase	Descripción	Inicio	Fin
Anteproyecto	Definición de ideas y entrega inicial	02/03/26	08/03/26
Análisis	Definición de requisitos y objetivos	09/03/26	15/03/26
Diseño	Arquitectura, base de datos y diagramas	16/03/26	22/03/26
Implementación	Desarrollo del código	23/03/26	10/04/26
Pruebas	Verificación del funcionamiento	11/04/26	15/04/26
Documentación	Redacción de la memoria	02/03/26	15/04/26

La planificación inicial sufrió modificaciones tras la reunión con el tutor, en la que se descartó el código base inicial y se comenzó un desarrollo completamente nuevo incorporando tecnologías como MongoDB y FastAPI, lo que amplió el alcance del

proyecto respecto a la idea original.

2.3. Ciclo de vida

El ciclo de vida seguido ha sido el modelo en cascada, que consta de las siguientes fases secuenciales: Requisitos, Diseño, Implementación, Pruebas y Mantenimiento. Cada fase produjo un resultado que sirvió de entrada para la siguiente.

Aunque el modelo es secuencial, durante el desarrollo fue necesario volver a fases anteriores puntualmente, especialmente tras la reunión con el tutor en la que se redefinieron los requisitos y se replanteó el diseño completo de la aplicación.

3. OBJETIVOS

El objetivo general del proyecto es desarrollar un centro comercial virtual funcional en Python que permita simular una experiencia de compra online completa.

Los objetivos específicos del proyecto son los siguientes:

- Implementar la navegación por plantas y tiendas del centro comercial.
- Desarrollar un carrito de la compra funcional con gestión de stock.
- Implementar la confirmación de compras y el historial de pedidos.
- Crear un modo administrador protegido por contraseña para gestionar productos.
- Persistir los datos mediante MongoDB como base de datos principal y ficheros JSON como copia de seguridad.
- Desarrollar una API REST con FastAPI que exponga todos los endpoints del sistema.
- Documentar el proyecto mediante una memoria técnica completa.

4. DESARROLLO DE LA APLICACIÓN

4.1. Análisis: requisitos

Requisitos funcionales

Los requisitos funcionales definen las acciones que el sistema debe ser capaz de realizar:

- El sistema debe permitir al usuario navegar por las plantas y tiendas del centro comercial.
- El sistema debe mostrar los productos disponibles en cada tienda con su nombre, precio y stock.
- El sistema debe permitir añadir productos al carrito indicando la cantidad deseada.
- El sistema debe controlar el stock disponible e impedir añadir más unidades de las existentes.
- El sistema debe permitir ver el contenido del carrito y el total a pagar.
- El sistema debe permitir confirmar la compra, registrarla en la base de datos y vaciar el carrito.
- El sistema debe permitir vaciar el carrito en cualquier momento.
- El sistema debe disponer de un modo administrador accesible mediante contraseña.
- El modo administrador debe permitir añadir, eliminar y modificar el precio de los productos.
- La API REST debe exponer todos los endpoints necesarios para operar con el sistema.

Requisitos no funcionales

Los requisitos no funcionales definen las características de calidad y restricciones técnicas del sistema:

- El sistema debe estar desarrollado en Python 3.
- La interfaz de usuario debe ser de consola, sencilla e intuitiva.
- Los datos deben persistir entre ejecuciones mediante MongoDB y ficheros JSON.
- La API REST debe estar desarrollada con FastAPI y ser accesible mediante Swagger UI.
- El código debe estar versionado en GitHub con un flujo de trabajo basado en ramas.
- La aplicación debe ejecutarse en sistemas Windows con MongoDB instalado localmente.

4.2. Diseño: arquitectura

Arquitectura de la aplicación

La aplicación sigue una arquitectura en tres capas claramente diferenciadas:

- Capa de presentación: formada por Centro.py, que gestiona la interfaz de consola y la interacción con el usuario, y por api.py, que expone la API REST con FastAPI y permite el acceso mediante Swagger UI desde el navegador.
- Capa de lógica de negocio: contiene toda la lógica de gestión de tiendas y productos, el carrito de la compra y el modo administrador. Esta lógica está implementada tanto en Centro.py como en api.py mediante funciones y endpoints.
- Capa de datos: formada por MongoDB, que actúa como base de datos principal almacenando las colecciones de productos, carrito y pedidos, y por los ficheros JSON (productos.json y carrito.json) que actúan como copia de seguridad sincronizada.

Base de datos

La base de datos utilizada es MongoDB, una base de datos NoSQL orientada a documentos. La base de datos se denomina unionshop y contiene las siguientes colecciones:

Colección	Descripción	Campos principales
productos	Almacena los productos organizados por planta y tienda.	planta, tienda, productos (array)
carrito	Almacena el carrito de la compra activo.	_id: 'carrito', productos (array)
pedidos	Almacena el historial de pedidos confirmados.	fecha, productos, total

4.3. Implementación: decisiones adoptadas y problemas encontrados

Herramientas utilizadas

- **Python3:** como lenguaje principal de programación, por ser el lenguaje trabajado durante el ciclo y por su facilidad para desarrollar aplicaciones de consola y APIs REST.
- **MongoDBAtlas**: como base de datos en la nube, donde se almacenan las colecciones de productos, carrito y pedidos. Se accede a ella desde Python mediante la librería pymongo.
- **FastAPI:** como framework para construir la API REST del proyecto, junto con Uvicorn como servidor ASGI para ejecutarla.
- **Rest Api:** REST API (cliente HTTP integrado en VS Code) para probar y verificar el correcto funcionamiento de los endpoints de la API durante el desarrollo.
- **pymongo**: librería de Python para conectarse a MongoDB.
- **Visual Studio Code:** entorno de desarrollo.
- **GitHub**: como repositorio de control de versiones, donde se ha ido subiendo el código del proyecto a lo largo del desarrollo.

Decisiones adoptadas

Durante el desarrollo se tomaron las siguientes decisiones técnicas relevantes:

- Se eligió MongoDB en lugar de ficheros JSON puros como sistema de persistencia principal, ya que permite consultas más eficientes y una mejor gestión de los datos a medida que el proyecto crecía.
- Se mantuvieron los ficheros JSON como copia de seguridad sincronizada con MongoDB, de forma que siempre existe un respaldo legible de los datos.
- Se desarrolló una API REST con FastAPI para permitir el acceso al sistema desde el navegador mediante Swagger UI, ampliando así las posibilidades de uso del proyecto más allá de la consola.
- Se utilizó el parámetro upsert=True en las operaciones de guardado del carrito en MongoDB, lo que permite crear el documento si no existe o actualizarlo si ya existe, evitando errores en la primera ejecución.

Problemas encontrados

- Tras la primera reunión con el tutor fue necesario descartar el código base inicial y comenzar el desarrollo desde cero con una arquitectura completamente nueva, lo que supuso un retraso en la planificación inicial.
- Se encontraron problemas con las rutas relativas de los ficheros JSON al ejecutar la aplicación desde distintos directorios, que se resolvieron utilizando rutas absolutas construidas con os.path.dirname y os.path.abspath.
- La sincronización entre MongoDB y los ficheros JSON requirió cuidado especial para garantizar que ambos sistemas estuvieran siempre actualizados tras cada operación de escritura.

8. BIBLIOGRAFÍA

Publicaciones

Chodorow, K. (2013). MongoDB: The Definitive Guide (2.ª ed.). O'Reilly Media.

Lutz, M. (2013). Learning Python (5.ª ed.). O'Reilly Media.

Enlaces web

Comisión Nacional de los Mercados y la Competencia. (2024). El comercio electrónico superó en España los 21.000 millones de euros en el primer trimestre de 2024. Recuperado de https://www.cnmc.es/prensa/datos-comercio-electronico-it24-20241004

FastAPI. (2024). FastAPI - documentación oficial. Recuperado de https://fastapi.tiangolo.com

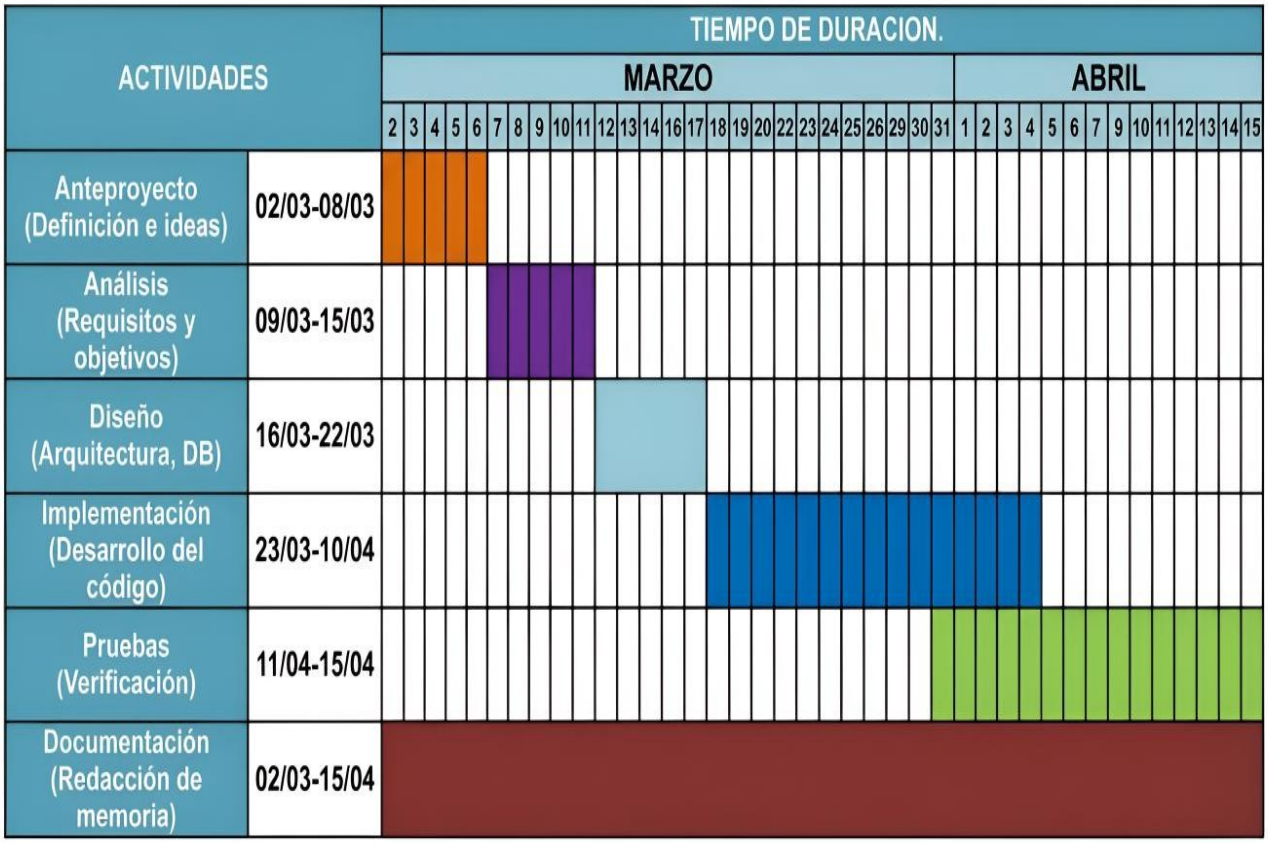
MongoDB. (2024). MongoDB Documentation. Recuperado de https://www.mongodb.com/docs

Python Software Foundation. (2024). Python 3 Documentation. Recuperado de https://docs.python.org/3

Ministerio de Educación, Formación Profesional y Deportes. (2024). Curso de Especialización en Desarrollo de aplicaciones en lenguaje Python. Recuperado de https://www.todofp.es/que-estudiar/familias-profesionales/informatica-comunicaciones/ce-lenguaje-phyton.html

https://code.visualstudio.com Para el entorno de desarrollo de VisualStudio code

DIAGRAMA DE GANTT: PROYECTO UNION-SHOP



Enlace a GitHub

https://github.com/hectorrostro/Union-Shop

Participantes:

Héctor José Rostro Almaraz

Jesús Tarifa Pozuelo

Ismael Sánchez Amores